

Table of Contents

 Resumo PAW — Programação em Ambiente Web (Exame).....	1
Índice	1
0. Conceitos Fundamentais da Web	3
1. Introdução ao HTML	6
2. Introdução ao CSS	8
3. Introdução ao JavaScript.....	10
4. JavaScript e HTML DOM.....	12
5. Introdução ao Backend com Node.js.....	15
6. Express Framework & Middleware	17
7. Node.js, Express & Bases de Dados.....	18
8. REST APIs.....	21
9. Client-Side Frameworks & TypeScript	23
10. Angular — Componentes & Serviços.....	25
11. Angular — Components, Forms, Deploy.....	27
12. Angular — Autenticação e Autorização	29
13. Deploy MEAN Stack Applications	31
14. Segurança em Aplicações Web.....	33
15. Exercícios Práticos de Debug.....	35
16. Armadilhas Comuns de Exame	37
 Resumo Rápido para Exame	38

Resumo PAW — Programação em Ambiente Web (Exame)

[!TIP] Este resumo cobre **todos os 13 PDFs** da pasta PPS. Está organizado pela ordem lógica do programa: Frontend → Backend → Full-Stack.

Índice

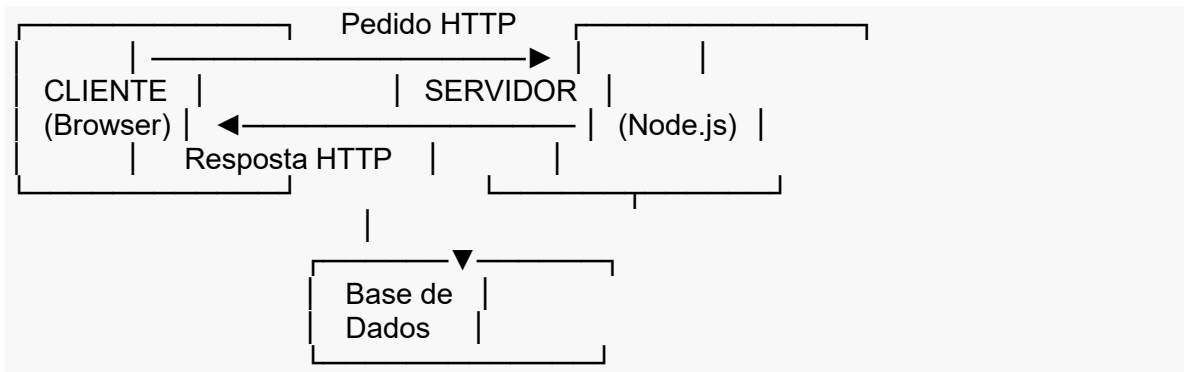
0. [Conceitos Fundamentais da Web](#)
1. [Introdução ao HTML](#)
2. [Introdução ao CSS](#)
3. [Introdução ao JavaScript](#)

4. [JavaScript e HTML DOM](#)
 5. [Introdução ao Backend com Node.js](#)
 6. [Express Framework & Middleware](#)
 7. [Node.js, Express & Bases de Dados](#)
 8. [REST APIs](#)
 9. [Client-Side Frameworks & TypeScript](#)
 10. [Angular — Componentes & Serviços](#)
 11. [Angular — Components, Forms, Deploy](#)
 12. [Angular — Autenticação e Autorização](#)
 13. [Deploy MEAN Stack Applications](#)
 14. [Segurança em Aplicações Web](#)
 15. [Exercícios Práticos de Debug](#)
 16. [Armadilhas Comuns de Exame](#)
-

0. Conceitos Fundamentais da Web

Modelo Cliente-Servidor ★

A arquitetura **cliente-servidor** é o modelo base de toda a web:



Cliente: - O **browser** do utilizador (Chrome, Firefox, etc.) - Executa código **HTML, CSS, JavaScript** (e frameworks como Angular) - **Inicia** a comunicação enviando pedidos HTTP - Responsável pela **interface com o utilizador** (frontend) - Não deve ter acesso direto a bases de dados nem a credenciais privadas

Servidor: - Aplicação que corre num computador remoto (ex: **Node.js + Express**) - **Recebe** pedidos HTTP, processa-os e **devolve** respostas - Acede a **bases de dados**, APIs externas e recursos protegidos - Responsável pela **lógica de negócio**, segurança e dados (backend) - Pode servir páginas HTML (SSR) ou dados JSON (API REST)

Fluxo típico: 1. Utilizador escreve URL ou clica num link → browser envia **pedido HTTP** ao servidor 2. Servidor processa o pedido (middleware → controller → model/BD) 3. Servidor devolve **resposta HTTP** (HTML, JSON, código de status) 4. Browser renderiza a resposta ao utilizador

Protocolo HTTP

HTTP (HyperText Transfer Protocol) é o protocolo de comunicação da web.

Estrutura de um pedido HTTP:

```
Método URL      Versão
GET  /products  HTTP/1.1
Host: www.exemplo.com
Content-Type: application/json
Authorization: Bearer token123
```

(corpo do pedido - vazio em GET, dados em POST/PUT)

Métodos HTTP (verbos):

Método	Propósito	Corpo?	Idempotente?
GET	Ler dados	Não	Sim
POST	Criar dados	Sim	Não
PUT	Atualizar dados	Sim	Sim
DELETE	Eliminar dados	Opcional	Sim

Códigos de Status HTTP :

Gama	Significado	Exemplos
1xx	Informacional	100 Continue
2xx	Sucesso 	200 OK , 201 Created, 204 No Content
3xx	Redirecionamento	301 Moved, 302 Found , 304 Not Modified
4xx	Erro do cliente 	400 Bad Request , 401 Unauthorized , 403 Forbidden , 404 Not Found
5xx	Erro do servidor 	500 Internal Server Error , 503 Service Unavailable

[!WARNING] **401 Unauthorized** = não autenticado (falta login). **403 Forbidden** = autenticado mas sem permissão (falta autorização). São diferentes!

HTTPS

- **HTTPS** = HTTP + SSL/TLS (criptação)
- Obrigatório em produção para proteger dados sensíveis (passwords, tokens)
- Em aplicações web, o transporte de informação HTTP deve ser feito via HTTPS

Diferenças Importantes

Conceito	NÃO é	É
HTML	Linguagem de programação	Linguagem de marcação (estrutura)
CSS	Linguagem de programação	Linguagem de estilos (apresentação)
JavaScript	Java	Linguagem de programação (lógica)
Node.js	Linguagem / Framework / Servidor web	Runtime para JavaScript
Express	Linguagem	Framework web para Node.js
Angular	Framework fullstack	Framework client-side (frontend)
NPM	Compilador / Runtime	Gestor de packages para JS
MongoDB	Base de dados relacional	Base de dados NoSQL (documentos)

Conceito	NÃO é	É
Mongoose	Base de dados	Driver/ODM para MongoDB em Node.js

[!IMPORTANT] **Para exame:** Saber explicar o modelo cliente-servidor, os métodos HTTP e seus códigos de status, a diferença entre HTTP e HTTPS, e as distinções da tabela acima (estas são armadilhas MUITO comuns).

1. Introdução ao HTML

O que é HTML?

- **HTML** = HyperText Markup Language
- Criado por **Tim Berners-Lee** (1991); versão atual: **HTML5** (2012/2014)
- Os browsers não mostram as tags, usam-nas para renderizar o conteúdo

Estrutura básica

```
<!DOCTYPE html>      <!-- Tipo de documento (HTML5) -->
<html lang="pt">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Título</title>
  </head>
  <body>
    <h1>Título Principal</h1>
    <p>Parágrafo</p>
  </body>
</html>
```

Conceitos-chave

Conceito	Descrição
Block-level	Inicia nova linha, ocupa toda a largura (<div>, <p>, <h1>, , <table>)
Inline	Não inicia nova linha, ocupa só o necessário (, <a>, ,)
Atributos	Declarados em minúsculas, valores entre aspas (src, alt, href, id, class)
Entidades	Caracteres especiais: < (<), > (>), & (&), (espaço)

Tags importantes

- **Cabeçalhos:** <h1> a <h6>
- **Links:** Texto
- **Imagens:**
- **Listas:** (não ordenada) / (ordenada) +
- **Tabelas:** <table>, <tr>, <th>, <td>, colspan, rowspan
- **Formulários:** <form action="/rota" method="POST">, <input>, <button>
 - type: text, password, radio, checkbox, submit
 - **Métodos GET vs POST** : NUNCA usar GET para dados sensíveis (como login/passwords), pois o GET expõe os dados no URL (visível no histórico). Usar sempre POST.

HTML Semântico (HTML5)

- <header>, <nav>, <main>, <section>, <article>, <aside>, <footer>, <figure>, <figcaption>

- Elementos multimédia: <video>, <audio>, <canvas>, <svg>

[!IMPORTANT] **Para exame:** Saber distinguir block vs inline, conhecer a estrutura básica de um documento HTML5, saber criar formulários e usar tags semânticas.

[!WARNING] **Armadilha:** HTML **NÃO** é uma linguagem de programação! É uma linguagem de **marcação** (markup). Não tem lógica, variáveis, condições, nem ciclos. Apenas descreve a **estrutura** do conteúdo. A formatação é CSS, a lógica é JavaScript.

2. Introdução ao CSS

O que é CSS?

- **CSS** = Cascading Style Sheets — descreve o estilo e apresentação do HTML
- Versões: CSS1 (1996), CSS2 (1998), **CSS3** (atual)

3 formas de adicionar CSS

1. **Inline**: `<tag style="color: red;">`
2. **Internal**: `<style>` dentro do `<head>`
3. **External**: `<link rel="stylesheet" href="ficheiro.css">` ★ (recomendado)

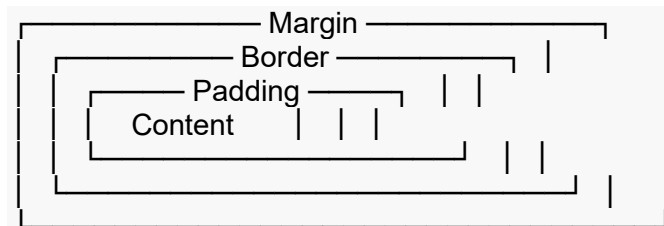
Sintaxe CSS

```
seletor {  
  propriedade: valor; /* cada declaração termina com ; */  
}
```

Seletores

Seletor	Sintaxe	Aplica-se a
Tag	<code>p { }</code>	Todas as tags <code><p></code>
Classe	<code>.nome { }</code>	Elementos com <code>class="nome"</code> (múltiplos)
ID	<code>#nome { }</code>	Elemento com <code>id="nome"</code> (único)

Box Model



- **Content**: texto e elementos
- **Padding**: espaço entre conteúdo e borda
- **Border**: entre padding e margin
- **Margin**: espaço exterior ao redor do elemento

Posicionamento

- **position: relative** — move em relação à posição original
- **position: absolute** — posição fixa na janela do browser
- **float**: left, right, none, inherit
- **clear**: controla elementos flutuantes ao lado
- **display**: none, block, inline

Web Responsiva

- **Viewport**: `<meta name="viewport" content="width=device-width, initial-scale=1.0">`

- **max-width: 100%** para imagens
- **Media Queries:**

```
@media (max-width: 480px) { /* phones */ }  
@media (max-width: 720px) { /* tablets */ }  
@media (min-width: 992px) { /* laptops */ }
```

- **Regra de ouro:** Nunca fazer scroll horizontal!

[!IMPORTANT] **Para exame:** Box Model é fundamental. Saber a diferença entre os 3 tipos de CSS, seletores (tag, classe, ID), e media queries.

[!WARNING] **Armadilha:** CSS **NÃO** é uma linguagem de programação! Apesar de suportar animações (@keyframes, transition) e até alguma lógica com calc(), CSS é uma linguagem **declarativa de estilos**. Não possui funções, variáveis com lógica condicional, nem estruturas de controlo de fluxo completas.

3. Introdução ao JavaScript

Características

- Linguagem **interpretada**, **multi-paradigma** (OO, funcional, imperativa, eventos)
- **Não é Java!** — executada em runtimes (browser ou Node.js)
- **Case sensitive**

Variáveis e Tipos

```
var nome = "João";    // String
let idade = 20;       // Number (let = block-scoped)
const PI = 3.14;      // Constante (ES6)
var ativo = true;     // Boolean
var vazio = null;     // Null
var x;               // Undefined
var lista = [1, 2, 3]; // Array
var obj = { nome: "Ana" }; // Object
```

[!WARNING] **var** tem scope de **função** (e sofre hoisting). **let** tem scope de **bloco**. **const** não pode ser reatribuído.

Operadores de Comparação

Operador	Significado
==	Igual (valor)
===	Igual (valor E tipo) ★
!=	Diferente
!==	Diferente (valor OU tipo)

Estruturas de Controlo

```
// Condicional
if (cond) { } else { }
(cond) ? valorTrue : valorFalse;
```

```
// Ciclos
for (let i = 0; i < 10; i++) { }
while (cond) { }
do { } while (cond);
```

Funções

```
// Function Declaration (sofre hoisting)
function soma(a, b) { return a + b; }
```

```
// Function Expression
const soma = function(a, b) { return a + b; };
```

```
// Arrow Function (ES6)
const soma = (a, b) => a + b;
```

- Argumentos primitivos passados **por valor**, objetos **por referência**
- Sem verificação de tipos nem número de argumentos

Objetos e Classes (ES6)

// Objeto literal

```
const pessoa = { nome: "Ana", idade: 25 };
```

// Classe ES6

```
class Animal {
  constructor(nome) { this.nome = nome; }
  falar() { console.log(this.nome + ' faz som'); }
}
class Cão extends Animal {
  falar() { console.log(this.nome + ' ladra'); }
}
```

Promises (ES6)

```
const promise = new Promise((resolve, reject) => {
  // operação assíncrona
  if (sucesso) resolve(resultado);
  else reject(erro);
});
promise.then(res => {}).catch(err => {});
```

- Estados: **pending** → **fulfilled** ou **rejected**

Boas Práticas

- Evitar variáveis globais
- Declarar variáveis no topo
- Usar {} em vez de new Object(), [] em vez de new Array()
- Usar === em vez de ==

[!IMPORTANT] **Para exame:** Diferença var/let/const, == vs ===, hoisting, scope, funções (declaration vs expression vs arrow), Promises, classes ES6.

4. JavaScript e HTML DOM

O que é o DOM?

- **DOM** = Document Object Model — interface padrão (W3C) para aceder e manipular documentos HTML
- O HTML DOM define elementos HTML como **objetos** com propriedades, métodos e eventos

Aceder a Elementos

Método	Descrição
document.getElementById(id)	Por ID (único)
document.getElementsByTagName(tag)	Por tag (coleção)
document.getElementsByClassName(classe)	Por classe (coleção)
document.querySelector(seletor)	Primeiro match CSS
document.querySelectorAll(seletor)	Todos os matches CSS

Manipular Elementos

// Alterar conteúdo

```
element.innerHTML = "Novo conteúdo";
```

// Alterar atributo

```
element.setAttribute("class", "destaque");
```

// Alterar estilo CSS

```
element.style.color = "red";
```

// Criar/remover elementos

```
const novo = document.createElement("p");
```

```
document.body.appendChild(novo);
```

```
document.body.removeChild(elemento);
```

Eventos

// Via propriedade

```
document.getElementById("btn").onclick = function() { };
```

// Via addEventListener (recomendado)

```
element.addEventListener("click", function() { });
```

```
element.removeEventListener("click", handler);
```

Relação entre Nodos

- parentNode, childNodes, firstChild, lastChild, nextSibling, previousSibling

Validação de Formulários

- checkValidity() — verifica restrições HTML
- setCustomValidity() — mensagens de erro personalizadas

Web APIs do Browser

- **window**: screen, location, history, timings, cookies
- **Storage API**: localStorage (persistente) e sessionStorage (sessão)

```
localStorage.setItem("chave", "valor");  
localStorage.getItem("chave");  
localStorage.removeItem("chave");
```

AJAX ★

- **AJAX** = Asynchronous JavaScript And XML
- Permite atualizar partes da página **sem recarregar** toda a página
- Usa o objeto **XMLHttpRequest**

```
const xhttp = new XMLHttpRequest();  
xhttp.onreadystatechange = function() {  
  if (this.readyState == 4 && this.status == 200) {  
    // resposta em this.responseText  
  }  
};  
xhttp.open("GET", "url", true); // true = assíncrono  
xhttp.send();
```

readyState	Estado
0	Não inicializado
1	Conexão estabelecida
2	Pedido recebido
3	A processar
4	Completo — resposta pronta ★

JSON

- **JSON** = JavaScript Object Notation — formato para serializar/transmitir dados
- JSON.parse(string) — converte string JSON → objeto JavaScript
- JSON.stringify(obj) — converte objeto → string JSON
- Tipos suportados: string, number, object, array, boolean, null

localStorage vs sessionStorage

Característica	localStorage	sessionStorage
Persistência	Permanente (sobrevive ao fechar browser)	Apenas durante a sessão/tab
Scope	Mesmo domínio, qualquer tab	Apenas a tab atual
Capacidade	~5-10 MB	~5-10 MB
Uso típico	Tokens, preferências, contadores	Dados temporários de formulário

[!IMPORTANT] **Para exame**: Saber manipular DOM (getElementById, innerHTML, eventos), AJAX (XMLHttpRequest, readyState), JSON

(parse/stringify), localStorage vs sessionStorage. getElementById retorna **UM** elemento (o primeiro), IDs devem ser **únicos** no HTML.

5. Introdução ao Backend com Node.js

O que é Node.js?

- **Runtime** JavaScript para executar JS fora do browser (servidor/desktop)
- **Não é um servidor web** por si só — é preciso codificar um servidor HTTP
- **Single-threaded** com I/O assíncrono (não bloqueante)
- Baseado no motor **V8** (Chrome)

Callbacks

- Função chamada **assincronamente** quando uma tarefa é concluída
- Modelo **não bloqueante**: Node.js pode processar outros pedidos enquanto espera I/O

```
// Bloqueante (síncrono)
const data = fs.readFileSync('file.txt');
console.log(data);
console.log("Fim");

// Não bloqueante (assíncrono) ★
fs.readFile('file.txt', function(err, data) {
  console.log(data);
});
console.log("Fim"); // Executa antes do callback!
```

Encadeamento de Operações Assíncronas

1. **Encadear funções** (callback hell)
2. **Promises** — .then().catch()
3. **Async/Await** — async function() { await ... } ★

Thread Starvation

- Funções síncronas pesadas (ex: fibonacci) bloqueiam o event-loop
- O timeout/callbacks ficam atrasados

NPM e package.json ★

- **NPM (Node Package Manager)**: Gestor de dependências.
- **package.json**: Ficheiro crucial que guarda as **dependências** (packages) da aplicação e os **scripts** executáveis (como "start": "node app.js").

```
npm init          # Inicializa projeto (cria package.json)
npm install pacote --save # Instala dependência e guarda no package.json
npm start         # Executa o script "start" do package.json
```

Módulos

```
// Importar módulo
const http = require('http');
const fs = require('fs');

// Criar módulo custom (CommonJS)
// ficheiro meuModulo.js
```

```
exports.funcao = function() { return "olá"; };
```

```
// Usar módulo custom
```

```
const mod = require('./meuModulo');
```

```
mod.funcao();
```

Módulos Importantes

- **fs** — sistema de ficheiros (readFile, writeFile, appendFile, open)
- **url** — parsing de URLs
- **http** — criar servidor HTTP básico

[!IMPORTANT] **Para exame:** Diferença entre I/O bloqueante vs não bloqueante, callbacks, Promises vs Async/Await, NPM (init, install, start), require/exports.

6. Express Framework & Middleware

O que é Express?

- **Framework web** minimal e flexível para Node.js
- Facilita routing, middleware, template engines
- Agnóstica em relação a design patterns

Iniciar Projeto Express

```
npm install -g express-generator
express --view ejs myapp # Gera projeto com EJS
cd myapp
npm install
npm start
```

- **Nota:** O express-generator cria automaticamente uma pasta genérica chamada **views** onde os templates EJS são guardados por defeito.

Request e Response ★

Request (req): | Propriedade | Descrição | |-----|-----| | req.query | Query strings (?key=val) | | req.params | Parâmetros de rota (:id) | | req.body | Corpo do pedido (POST) | | req.cookies | Cookies | | req.path, req.ip | Caminho e IP |

Response (res): | Método | Descrição | |-----|-----| | res.status(code) | Define código HTTP | | res.send(body) | Envia resposta | | res.json(obj) | Envia JSON | | res.render(view, data) | Renderiza template | | res.redirect(url) | Redireciona | | res.cookie(name, val) | Define cookie |

Routing

```
// Métodos HTTP
app.get('/rota', (req, res) => { });
app.post('/rota', (req, res) => { });
app.put('/rota/:id', (req, res) => { });
app.delete('/rota/:id', (req, res) => { });
```

```
// Parâmetros de rota
app.get('/produto/:id', (req, res) => {
  const id = req.params.id;
});
```

```
// Router separado (modular)
const router = express.Router();
router.get('/', handler);
app.use('/users', router);
```

Ficheiros Estáticos

```
app.use('/public', express.static('public'));
```

Módulos úteis

- **body-parser** — parse do body de pedidos HTTP

- **cookie-parser** — parse de cookies
- **multer** — upload de ficheiros (multipart)

Template Engine — EJS

```
npm install ejs --save
```

- Tags EJS:
 - `<% %>` — código JS (sem output)
 - `<%= %>` — output (HTML escaped)
 - `<%- %>` — output (sem escape)
 - `<%# %>` — comentário

```
// No Express
app.set('view engine', 'ejs');
res.render('pagina', { nome: "João", items: [1,2,3] });
```

Padrão MVC

- **Model** — dados
- **View** — apresentação (templates EJS)
- **Controller** — lógica que liga Model a View

Middleware

- Funções executadas **entre o pedido e a resposta**
- Têm acesso a req, res e next()

```
// Middleware ao nível da aplicação
app.use(function(req, res, next) {
  console.log('Pedido recebido');
  next(); // IMPORTANTE: chamar next()!
});
```

```
// Middleware de erros (4 parâmetros)
app.use(function(err, req, res, next) {
  res.status(500).send('Erro!');
});
```

Tipos de middleware: 1. **Aplicação** — app.use() 2. **Routing** — router.use() 3. **Erros** — 4 parâmetros (err, req, res, next) 4. **Built-in** — express.static, express.json, express.urlencoded 5. **Third-party** — módulos npm (body-parser, multer, cors)

[!WARNING] A **ordem** do middleware importa! E nunca esquecer de chamar next() se não for o fim do ciclo.

[!IMPORTANT] **Para exame:** Saber criar rotas (GET/POST/PUT/DELETE), req.params vs req.query vs req.body, middleware (o que é, tipos, next()), MVC, EJS tags.

7. Node.js, Express & Bases de Dados

Armazenamento de Dados

- **Ficheiros** (JSON, XML) — simples, bom para imagens/vídeos

- **BD Relacional** (SQL) — transações, alta consistência
- **BD NoSQL** — flexível, escalável, grandes volumes
- **Serviços cloud** (Firebase, etc.)

CRUD

- **Create**, **Read**, **Update**, **Delete**

ACID (BD Relacionais)

- **A**tomicidade, **C**onsistência, **I**solamento, **D**urabilidade
- BD NoSQL podem **não** cumprir ACID (consistência eventual)

SQL vs NoSQL

Aspecto	SQL (Relacional)	NoSQL
Schema	Rígido, bem definido	Flexível, sem schema
Linguagem	SQL	Específica de cada BD
Transações	ACID	Consistência eventual
Escalabilidade	Vertical	Horizontal
Exemplos	MySQL, PostgreSQL, Oracle	MongoDB, CouchDB, Firebase

MongoDB ★

- BD **NoSQL**, orientada a **documentos** (formato JSON/BSON)
- Parte da **MEAN Stack** (MongoDB, Express, Angular, Node)

Terminologia: | SQL | MongoDB | |——|———| | Tabela | Coleção | | Linha | Documento | | Campo | Field |

Mongoose (Driver MongoDB para Node.js)

npm install mongoose --save

```
const mongoose = require('mongoose');

// Conexão
mongoose.connect('mongodb://localhost/mydb');

// Schema + Model
const produtoSchema = new mongoose.Schema({
  nome: String,
  preco: Number,
  quantidade: Number
});
const Produto = mongoose.model('Produto', produtoSchema);

// CRUD
Produto.find({}, callback); // Read
new Produto({nome: "X"}).save(callback); // Create
Produto.findByIdAndUpdate(id, data, callback); // Update
Produto.findByIdAndDelete(id, callback); // Delete
```

MySQL com Node.js

npm install mysql --save

```
const mysql = require('mysql');  
const con = mysql.createConnection({ host, user, password, database });  
con.query("SELECT * FROM tabela", callback);
```

ORM (Object Relational Mapping)

- Mapeia objetos do código para tabelas da BD
- Exemplos: **Sequelize**, Bookshelf, Objection.js

Autenticação vs Autorização

- **Autenticação** — “Quem és?” (identificar o utilizador)
- **Autorização** — “O que podes fazer?” (permissões)
- Usar **Passport.js** para autenticação, **ACL** para autorização

[!IMPORTANT] **Para exame:** Diferença SQL vs NoSQL, ACID, CRUD, Mongoose (schema/model/operações), diferença autenticação vs autorização.

8. REST APIs

O que é REST?

- **REST** = Representational State Transfer
- Padrão arquitetural baseado em HTTP para criar web services
- Definido por **Roy Fielding** (2000)
- **Formatos de Dados**: A informação é trocada tipicamente em texto estruturado (**JSON** ou **XML**), e não em formato binário, o que permite o consumo fácil por aplicações frontend.

Propriedades REST

- Arquitetura **cliente-servidor**
- **Stateless** (sem estado entre pedidos)
- **Cacheable**
- **Interface uniforme**
- Sistema em **camadas**

Métodos HTTP → CRUD ★

Método	Operação CRUD	Exemplo
GET	Read	GET /products
POST	Create	POST /products
PUT	Update	PUT /product/:id
DELETE	Delete	DELETE /product/:id

Exemplo de API REST em Express

```
// GET todos os produtos
router.get('/products', productController.getAll);

// GET produto por ID
router.get('/product/:id', productController.getById);

// POST novo produto
router.post('/products', productController.create);

// PUT atualizar produto
router.put('/product/:id', productController.update);

// DELETE remover produto
router.delete('/product/:id', productController.delete);
```

CORS (Cross-Origin Resource Sharing)

```
npm install cors --save
```

```
const cors = require('cors');
app.use(cors()); // Permite pedidos de outros domínios
```

JWT (JSON Web Token) ★

- Standard aberto (RFC 7519) para transmitir informação segura
- Assinado com HMAC ou RSA/ECDSA

```
npm install jsonwebtoken --save
```

```
const jwt = require('jsonwebtoken');
```

```
// Criar token
```

```
const token = jwt.sign({ userId: user._id }, 'segredo', { expiresIn: '1h' });
```

```
// Verificar token (middleware)
```

```
jwt.verify(token, 'segredo', (err, decoded) => { });
```

OpenAPI / Swagger

- Padrão para **documentar** APIs REST
- **swagger-ui-express** — gera documentação interativa

```
npm install swagger-ui-express --save
```

- Ficheiro swagger.json define endpoints, parâmetros, respostas

Testes de API

- **Postman** — ferramenta para testar APIs REST (enviar GET, POST, PUT, DELETE)

[!IMPORTANT] **Para exame:** Métodos HTTP ↔ CRUD, estrutura de uma API REST, CORS (porquê e como resolver), JWT (o que é, como funciona), Swagger.

9. Client-Side Frameworks & TypeScript

Frameworks do Lado do Cliente

- Permitem criar **SPA** (Single Page Application)
- Exemplos: **Angular**, React, Vue, Meteor
- Usam o conceito **App Shell** — estrutura descarregada uma vez, dados atualizados via API

SPA vs MPA

SPA	MPA
Uma única página, conteúdo dinâmico	Múltiplas páginas, recarrega tudo
Melhor UX, mais rápido após carga inicial	Mais simples, melhor SEO nativo
Angular, React, Vue	Sites tradicionais

TypeScript ★

- **Superset** de JavaScript (tudo que é JS é válido em TS)
- Criado pela **Microsoft** (Anders Hejlsberg, designer de C#)
- Adiciona: **tipos estáticos**, POO mais robusta, verificação de erros em compilação
- Ficheiros .ts → compilados para .js com tsc

```
// Tipos
let nome: string = "João";
let idade: number = 25;
let ativo: boolean = true;
let qualquer: any = "qualquer coisa";

// Funções tipadas
function soma(a: number, b: number): number {
  return a + b;
}

// Classes
class Pessoa {
  nome: string;
  constructor(nome: string) { this.nome = nome; }
}

npm install -g typescript
tsc ficheiro.ts    # Compila para JS
node ficheiro.js   # Executa
```

Introdução ao Angular

- Criado pela **Google** (2012), escrito em **TypeScript**
- Baseado no padrão **MVC**
- Blocos fundamentais:
 - **NgModules** — agrupam componentes e serviços

- **Components** — classe TS + template HTML + estilo CSS
- **Services** — lógica partilhada entre componentes (@Injectable)
- **Templates** — HTML com diretivas Angular (data binding bidirecional)
- **Router** — navegação entre views (SPA)
- **DI (Dependency Injection)** — injeta serviços nos componentes

```
npm install -g @angular/cli  
ng new my-app          # Cria projeto  
npm start              # ou: ng serve --open  
ng generate component nome # Cria componente (4 ficheiros)
```

[!IMPORTANT] **Para exame:** SPA vs MPA, TypeScript (tipos, compilação), conceitos Angular (NgModule, Component, Service, Router, DI).

10. Angular — Componentes & Serviços

Estrutura de um Projeto Angular

- Pasta `src/app/` contém os ficheiros principais
- Cada **componente** tem 4 ficheiros:
 - `*.component.ts` — lógica (TypeScript)
 - `*.component.html` — template (HTML)
 - `*.component.css` — estilos
 - `*.component.spec.ts` — testes

Criar Componentes e Serviços

```
ng g component product-detail # Gera componente
ng g service rest # Gera serviço
```

Routing

```
// app-routing.module.ts
const routes: Routes = [
  { path: 'products', component: ProductListComponent },
  { path: 'product/:id', component: ProductDetailComponent },
];
```

- **ActivatedRoute** — obter parâmetros do URL
- **Router** — redirecionar programaticamente

Serviços (Services)

- Usam **HttpClient** para comunicar com APIs REST
- Usam **RxJS** (Observables, Subscribe) para programação reativa

```
@Injectable({ providedIn: 'root' })
export class RestService {
  constructor(private http: HttpClient) {}

  getProducts(): Observable<any> {
    return this.http.get('http://api/products');
  }
}
```

Ciclo de Vida do Componente ★

Hook	Quando
<code>ngOnInit()</code>	Após inicialização (1 vez) — o mais usado
<code>ngOnChanges()</code>	Quando inputs mudam
<code>ngOnDestroy()</code>	Antes de destruir (cleanup)
<code>ngDoCheck()</code>	A cada deteção de mudança

Data Binding

Tipo	Sintaxe	Direção
Interpolation	<code>{{ variavel }}</code>	Component → View

Tipo	Sintaxe	Direção
Property binding	[propriedade]="valor"	Component → View
Event binding	(evento)="handler()"	View → Component
Two-way binding	[(ngModel)]="variavel"	Bidirecional ★

app.module.ts

- Declara todos os **componentes**, **imports** (módulos), **providers** (serviços)
[!IMPORTANT] **Para exame:** Criar componentes/serviços, routing com parâmetros, HttpClient + Observables, ciclo de vida (ngOnInit), tipos de data binding.
-

11. Angular — Components, Forms, Deploy

Comunicação entre Componentes

- Usar **serviços** para manter estado partilhado entre componentes
- **BehaviorSubject** (RxJS) — padrão publish/subscribe para propagar mudanças

```
// No serviço
private notesSource = new BehaviorSubject<Note[]>([]);
notes$ = this.notesSource.asObservable();

addNote(note: Note) {
  const current = this.notesSource.value;
  this.notesSource.next([...current, note]);
}
```

Persistência com localStorage

```
// Guardar
localStorage.setItem('notes', JSON.stringify(notes));

// Ler
const saved = JSON.parse(localStorage.getItem('notes') || '[]');
```

Formulários em Angular

Tipo	Característica
Template-driven	Simples, diretivas no HTML (ngModel)
Reactive	Mais controlo, definido no TS

Validação de Formulários

```
<input name="nome" ngModel required minlength="3" #nome="ngModel">
<div *ngIf="nome.invalid && nome.touched">
  <span *ngIf="nome.errors?.required">Campo obrigatório</span>
  <span *ngIf="nome.errors?.minlength">Mín. 3 caracteres</span>
</div>
```

Angular Material

- Biblioteca de **componentes UI** prontos para Angular

```
ng add @angular/material
```

- Componentes: mat-card, mat-button, mat-input, mat-select, MatDialog
- Importar módulos em app.module.ts

MatDialog (Diálogos)

- Componente **dinâmico** que comunica com o componente invocador
- Recebe inputs e devolve outputs
- Declarar em entryComponents no module

Deploy Angular

```
ng build # Gera build de produção em dist/
```

- Resultado: ficheiros **estáticos** (HTML/CSS/JS) na pasta dist/
- Opções de deploy: **GitLab Pages**, GitHub Pages, AWS, Azure, Heroku
- Para GitLab Pages: criar ficheiro .gitlab-ci.yml com script de build

[!IMPORTANT] **Para exame:** BehaviorSubject para estado partilhado, validação de formulários (template-driven), Angular Material (como importar e usar), ng build.

12. Angular — Autenticação e Autorização

Autenticação vs Autorização (revisão)

- **Autenticação** — verificar identidade (login)
- **Autorização** — verificar permissões (admin vs guest)

Backend (Express + JWT)

1. Instalar jsonwebtoken
2. Definir **segredo** em authconfig.js
3. Criar **modelo User** com Mongoose
4. Criar **controlador de autenticação**:
 - Login → verifica credenciais → gera token JWT
 - Middleware → verifica token nos headers de cada pedido protegido

Frontend (Angular)

1. **Serviço de autenticação** — comunica com API REST de login
2. **Guardar token** no localStorage
3. **Route Guards** — interface CanActivate

```
// auth.guard.ts
canActivate(): boolean {
  if (localStorage.getItem('token')) return true;
  this.router.navigate(['/login']);
  return false;
}

// routing
{ path: 'admin', component: AdminComponent, canActivate: [AuthGuard] }
```

4. **Interceptors** — adicionam token a **todos** os pedidos HTTP automaticamente

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = localStorage.getItem('token');
    const cloned = req.clone({
      headers: req.headers.set('Authorization', 'Bearer ' + token)
    });
    return next.handle(cloned);
  }
}
```

Reutilização de Componentes

- Passar dados entre componentes com **@Input()**

```
// No componente filho
@Input() produto: Produto;

// No template pai
<app-produto-detalle [produto]="produtoSelecionado"></app-produto-detalle>
```

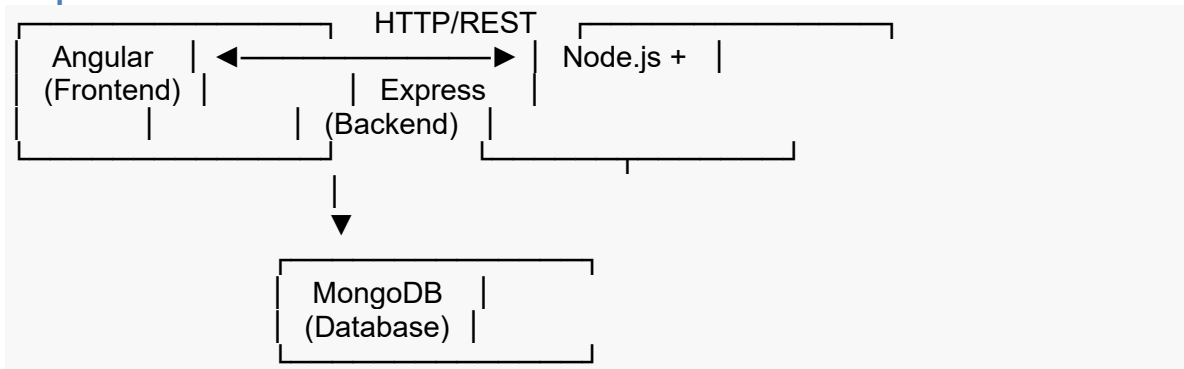
Angular Material (revisão)

- `ng add @angular/material`
- Importar módulos necessários em `app.module.ts`
- Usar diretamente nos templates: `<mat-toolbar>`, `<mat-card>`, etc.

[!IMPORTANT] **Para exame:** Fluxo completo de autenticação JWT (backend + frontend), Route Guards (`CanActivate`), Interceptors, `@Input()` para passar dados entre componentes.

13. Deploy MEAN Stack Applications

Arquitetura MEAN Stack



Regras de Segurança ★

- **Nunca** ligar Angular diretamente à BD (código fica exposto no cliente!)
- Usar o backend como **intermediário** (broker) para:
 - Acesso a BD
 - APIs externas que requerem credenciais
 - Autenticação/Autorização
- Pedidos a recursos públicos podem ser feitos diretamente pelo frontend

Opções de Deploy

Componente	Onde fazer deploy
Angular (frontend)	GitLab/GitHub Pages (estático), Servidor próprio
Node.js/Express (backend)	Heroku, AWS, Azure, Google Cloud
MongoDB	MongoDB Atlas (cloud), servidor próprio

Deploy do Backend (Heroku + MongoDB Atlas)

MongoDB Atlas (BD na cloud): 1. Criar conta e cluster 2. Permitir acesso de qualquer IP 3. Criar utilizador da BD 4. Obter URL de conexão

Heroku (servidor Node.js):

```
git init
git add .
git commit -m "initial commit"
heroku login
heroku git:remote -a nome-projeto
git push heroku master
```

- Heroku deteta Node.js automaticamente e executa `npm install + npm start`

Angular + Node.js no mesmo servidor

```
// No app.js do Express
app.use(express.static('dist/angular-app')); // Pasta com build Angular
```

```
// Para todas as rotas não-API, servir o Angular
app.get('*', (req, res) => {
  res.sendFile(path.join(__dirname, 'dist/angular-app/index.html'));
});
```

- Editar package.json:

```
"start": "ng build && node app.js"
```

[!IMPORTANT] **Para exame:** Arquitetura MEAN Stack, porquê separar frontend/backend, segurança (nunca expor BD ao cliente), deploy com Heroku + MongoDB Atlas, servir Angular a partir de Express.

14. Segurança em Aplicações Web

Validação de Dados ★

[!CAUTION] A validação de dados **DEVE** ser feita em **AMBOS** os lados — frontend **E** backend!

Onde	Porquê	Como
Frontend	Melhor UX (feedback imediato)	Atributos HTML (required, minlength), validação Angular (ngModel, Validators)
Backend	Segurança (o utilizador pode contornar o frontend!)	Middleware Express, validação no controller, Mongoose validators

Porquê validar no backend? - O utilizador pode **desativar JavaScript** no browser - Pode enviar pedidos diretamente com **Postman** ou **curl** - Pode **manipular** o HTML/JS no DevTools do browser - Confiar apenas no frontend = **falha de segurança grave**

Princípios de Segurança Web

1. **Nunca** expor credenciais no código do frontend (passwords, API keys, DB connection strings)
2. **Nunca** ligar o frontend diretamente a uma base de dados
3. **Sempre** usar **HTTPS** em produção
4. **Sempre** validar e sanitizar inputs no servidor
5. Usar o backend como **broker/intermediário** para APIs externas com credenciais
6. Guardar passwords de forma **hashed** (nunca em texto limpo)
7. Usar **JWT** com segredo forte e tempo de expiração

CORS (Cross-Origin Resource Sharing)

- Browsers bloqueiam pedidos a domínios diferentes por segurança (**Same-Origin Policy**)
- Para permitir acesso cross-origin, o **servidor** deve enviar headers específicos
- Em Express: `npm install cors` → `app.use(cors())`
- Pode ser configurado para permitir apenas domínios específicos

Autenticação vs Autorização (Resumo Definitivo)

	Autenticação	Autorização
Pergunta	“Quem és tu?”	“O que podes fazer?”
Quando	No login	Após login, em cada pedido
Como	Username/password → token JWT	Verificar role/permisões do token
Exemplo	<code>jwt.sign({id, role}, secret)</code>	<code>if (decoded.role !== 'ADMIN') return 403</code>

	Autenticação	Autorização
Código HTTP	401 Unauthorized	403 Forbidden
Módulo	jsonwebtoken, passport	Guards (Angular), ACL, roles

[!IMPORTANT] **Para exame:** Saber distinguir autenticação de autorização, porquê validar em ambos os lados, CORS, e princípios de segurança (nunca expor BD ao cliente).

15. Exercícios Práticos de Debug

Exercício 1 — Encontrar erros em HTML/JS

```
<!DOCTYPE html>
<html>
<body>
  <h1>Contador</h1>
  <p>Valor: <span id="valor">0</span></p>
  <button>Incrementar</button>
  <script>
    function incrementar() {
      contador++;
      document.getElementById(valor).innerHTML = contador;
    }
  </script>
</body>
</html>
```

Erros: 1. ✗ contador nunca é **declarado** → Adicionar `var contador = 0;` 2. ✗ `getElementById(valor)` — `valor` é variável `undefined` → Deve ser **string**: `getElementById("valor")` 3. ✗ O botão **não tem evento** associado → Adicionar `onclick="incrementar()"`

Exercício 2 — Analisar middleware Express

```
const verify = function(req, res, next) {
  var token = req.headers['x-access-token'];
  if (!token)
    return res.status(403).send({ message: 'No token' });
  jwt.verify(token, secret, (err, decoded) => {
    if (err) return res.status(500).send({ message: 'Token inválido' });
    if (decoded.role !== 'ADMIN') return res.status(403).send({ message: 'Sem permissão' });
  });
  req.userId = decoded.id;
  next();
};
```

Análise: - É um **middleware** Express (tem `req`, `res`, `next`) - **Não** é Angular (não tem decorators, não é TypeScript) - Faz **autenticação** (verifica se token existe e é válido) **E autorização** (verifica role ADMIN) - Chama `next()` apenas se autenticação e autorização passam - O middleware **não** é o último handler — passa ao próximo via `next()`

Exercício 3 — Identificar tecnologia por código

Código	Tecnologia	Pistas
@Injectable(), Observable, HttpClient	Angular (Service)	Decorators TS, RxJS
req, res, next, app.use()	Express (Middleware)	Parâmetros de middleware

Código	Tecnologia	Pistas
mongoose.Schema, mongoose.model	Mongoose (MongoDB)	Schema/Model pattern
document.getElementById(), addEventListener	JavaScript DOM	API do browser
res.render('view', data)	Express + Template Engine	Server-side rendering
jwt.sign(), jwt.verify()	JWT (jsonwebtoken)	Tokens de autenticação
ng generate component	Angular CLI	Comando ng
npm install, npm start	NPM	Gestor de packages

Exercício 4 — localStorage para persistência

Problema: Como manter dados entre recarregamentos da página usando apenas HTML/JS?

```
// 1. Ao carregar a página, ler dados guardados
var dados = JSON.parse(localStorage.getItem('meusDados')) || valorPadrão;

// 2. Mostrar dados no HTML
document.getElementById('elemento').textContent = dados;

// 3. Ao alterar dados, guardar no localStorage
function atualizar(novoDado) {
  dados = novoDado;
  document.getElementById('elemento').textContent = dados;
  localStorage.setItem('meusDados', JSON.stringify(dados));
}
```

Padrão: getItem → usar → alterar → setItem

[!IMPORTANT] **Para exame:** Saber encontrar erros comuns em HTML/JS (variáveis não declaradas, strings sem aspas, eventos em falta), identificar tecnologias por excerto de código, e usar localStorage para persistência.

16. Armadilhas Comuns de Exame

✗ Afirmações FALSAS frequentes

Afirmação falsa	Porquê é falsa
"HTML é uma linguagem de programação"	É de marcação (markup), não tem lógica
"CSS é uma linguagem de programação"	É de estilos (style), não é Turing-complete
"Node.js é uma linguagem de programação"	É um runtime (ambiente de execução). A linguagem é JavaScript
"Angular é uma framework fullstack"	É client-side (frontend). Fullstack = Angular + Express + MongoDB
"NPM cria componentes Angular"	NPM gere packages. Componentes são criados com ng generate (Angular CLI)
"NPM instala MongoDB"	NPM instala o driver (mongoose). MongoDB é instalado separadamente
"Express executa no browser"	Express é server-side (Node.js). O browser apenas envia pedidos HTTP
"getElementById retorna todos os elementos"	Retorna UM (o primeiro). Para vários: <code>getElementsByClassName</code> ou <code>querySelectorAll</code>
"Validação só no frontend é suficiente"	Não! Frontend pode ser contornado. Validar sempre no backend também
"Serviços Angular têm template HTML"	Não! Serviços são classes TS puras. Só componentes têm .html
"MongoDB pode correr no frontend"	Não! BD corre no servidor. Frontend usa localStorage ou API REST
"O JWT obriga a usar REST API"	Não! JWT pode ser usado com qualquer tipo de aplicação web
"REST só permite formato JSON"	Não! Permite e suporta XML (e não costuma ser binário)
"Formulários de login usam GET"	Perigoso/Falso! GET expõe a password no URL. Login exige POST

✓ Afirmações VERDADEIRAS frequentes

Afirmação verdadeira	Justificação
"Angular permite reutilização de componentes"	Sim, via <code>@Input()</code> , <code>selectors</code> , e serviços partilhados
"Serviços Angular podem ser injetados em múltiplos componentes"	Sim, via Dependency Injection (<code>@Injectable</code>)
"Express suporta template engines"	Sim: EJS, Pug, Mustache (<code>res.render()</code>)

Afirmação verdadeira	Justificação
“Mongoose permite aceder ao MongoDB”	Sim, é o driver/ODM Node.js para MongoDB
“npm start inicia um projeto”	Sim, executa o script “start” do package.json
“Angular usa Observables para comunicação assíncrona”	Sim, via RxJS (Observable, BehaviorSubject, subscribe)

Distinções Críticas

Conceito A	vs	Conceito B	Diferença-chave
var	vs	let	var = scope função + hoisting; let = scope bloco
==	vs	===	== compara valor (com coerção); === compara valor E tipo
innerHTML	vs	textContent	innerHTML interpreta HTML; textContent trata como texto
getElementById	vs	querySelectorAll	Retorna 1 elemento vs todos os matches
localStorage	vs	sessionStorage	Persistente vs apenas durante a sessão/tab
Frontend	vs	Backend	Browser (HTML/CSS/JS/Angular) vs Servidor (Node.js/Express)
Autenticação	vs	Autorização	“Quem és?” (401) vs “O que podes fazer?” (403)
ng	vs	npm	CLI Angular (componentes, build) vs Gestor de packages
SPA	vs	MPA	Uma página dinâmica vs múltiplas páginas com reload
SQL	vs	NoSQL	Schema rígido + ACID vs Flexível + consistência eventual
Component	vs	Service	Tem template HTML + lógica vs Apenas lógica (sem HTML)
Guard	vs	Interceptor	Protege rotas vs Modifica pedidos HTTP
req.params	vs	req.query	/rota/:id (URL path) vs ?key=val (query string)
Middleware	vs	Controller	Processa/filtra pedidos vs Lógica de negócio

Resumo Rápido para Exame

Conceitos Fundamentais

Tema	Conceitos-Chave
Web	Modelo cliente-servidor, HTTP (métodos, códigos), HTTPS
HTML	Tags semânticas, formulários, block vs inline, atributos, NÃO é

Tema	Conceitos-Chave
	programação
CSS	Seletores (tag, .classe, #id), Box Model, position, media queries, NÃO é programação
JavaScript	var/let/const, ===/==, hoisting, scope, Promises, async/await, classes ES6
DOM	getElementById (retorna UM!), innerHTML, addEventListener, createElement
AJAX/JSON	XMLHttpRequest, readyState, JSON.parse/stringify
Storage	localStorage (persistente) vs sessionStorage (sessão)
Node.js	Runtime JS (não é linguagem!), callbacks, I/O não-bloqueante, npm, require/exports
Express	Routing (GET/POST/PUT/DELETE), middleware (req/res/next), EJS, MVC
BD	SQL vs NoSQL, CRUD, ACID, Mongoose (MongoDB), Sequelize (MySQL/ORM)
REST API	Métodos HTTP ↔ CRUD, CORS, JWT, Swagger/OpenAPI, Postman
Segurança	Validação dupla (frontend+backend), HTTPS, CORS, JWT, nunca expor BD ao cliente
TypeScript	Superset JS, tipos estáticos, compilação tsc → js
Angular	Components (MVC), Services (DI), Routing, Observables, Guards, Interceptors, @Input
Deploy	MEAN Stack, ng build, Heroku, MongoDB Atlas, GitLab/GitHub Pages

Fluxo Completo de uma Aplicação MEAN

1. Utilizador interage com Angular (Frontend/SPA no Browser)
2. Guard verifica se utilizador pode aceder à rota
3. Componente Angular usa Serviço com HttpClient
4. Interceptor adiciona token JWT ao header do pedido HTTP
5. Express recebe pedido → Middleware verifica JWT → Controller
6. Controller interage com MongoDB via Mongoose (Model)
7. Resposta JSON volta pelo Express → Angular
8. Angular atualiza a View via data binding (Observable → subscribe)

O que cada ferramenta FAZ vs NÃO FAZ

- ✓ NPM: instala packages, gere dependências, executa scripts (npm start)
- ✗ NPM: NÃO cria componentes, NÃO instala MongoDB, NÃO compila TypeScript
- ✓ Angular CLI (ng): cria projetos, componentes, serviços, faz build
- ✗ Angular CLI: NÃO gere packages (isso é o npm), NÃO corre no servidor
- ✓ Express: routing, middleware, serve páginas/API, usa template engines

- ✗ Express: NÃO corre no browser, NÃO é uma linguagem
- ✓ Mongoose: liga Node.js ao MongoDB, define schemas/models, CRUD
- ✗ Mongoose: NÃO é uma base de dados (MongoDB é), NÃO instala MongoDB
- ✓ JWT: autenticação por tokens, assinatura digital, stateless
- ✗ JWT: NÃO obriga a usar REST, NÃO guarda sessões no servidor